

Absolut richtig mitgedacht! Wenn man die Tabelle später als Dokumentation öffnet oder weitergibt, muss natürlich sofort ersichtlich sein, welche Dateien da überhaupt verglichen wurden.

Ich habe die Export-Funktion in **Version 3.4** so angepasst, dass sie sich vor dem Generieren der .xls-Datei die aktuellen Dateipfade aus der Oberfläche zieht und sie elegant in die dunklen Kopfzeilen der Excel-Tabelle integriert.

Hier ist das Update. Einfach den Code komplett überschreiben:

```
import difflib
import webbrowser
import os
import json
import html
import tkinter as tk
from tkinter import filedialog
from http.server import HTTPServer, BaseHTTPRequestHandler

# --- App Konfiguration ---
APP_VERSION = "v3.4"

def waehle_datei_dialog(titel):
    root = tk.Tk()
    root.withdraw()
    root.attributes('-topmost', True)
    root.lift()
    root.focus_force()

    file_path = filedialog.askopenfilename(
        parent=root, title=titel,
        filetypes=[("Text/Code", "*.txt *.cfg *.py *.xml *.csv *.md
*.ini *.gcode"), ("Alle", "*.")]
    )
    root.destroy()
    return file_path

def get_file_info(file_path):
    if not file_path or not os.path.exists(file_path):
        return {"path": "", "content": [], "siblings": []}

    with open(file_path, 'r', encoding='utf-8', errors='replace') as
f:
        content = f.readlines()

    directory = os.path.dirname(file_path)
    valid_ext = ('.txt', '.ini', '.cfg', '.py', '.xml', '.csv', '.md',
'.json', '.gcode')
    siblings = []
    try:
        for f_name in os.listdir(directory):
```

```

        if f_name.lower().endswith(valid_ext):
            siblings.append({
                "name": f_name,
                "path": os.path.join(directory,
f_name).replace('\\', '/')
            })
        except Exception: pass

    siblings = sorted(siblings, key=lambda x: x['name'].lower())
    return {"path": file_path.replace('\\', '/'), "content": content,
"siblings": siblings}

def get_inline_diff(lines_left, lines_right):
    left_res, right_res = [], []
    if len(lines_left) != len(lines_right):
        return [html.escape(l) for l in lines_left], [html.escape(l)
for l in lines_right]

    for l, r in zip(lines_left, lines_right):
        sm = difflib.SequenceMatcher(None, l, r)
        out_l, out_r = "", ""
        for tag, i1, i2, j1, j2 in sm.get_opcodes():
            chunk_l = html.escape(l[i1:i2])
            chunk_r = html.escape(r[j1:j2])
            if tag == 'equal':
                out_l += chunk_l
                out_r += chunk_r
            else:
                if chunk_l: out_l += f"<span
class='char-diff'>{chunk_l}</span>"
                if chunk_r: out_r += f"<span
class='char-diff'>{chunk_r}</span>"
            left_res.append(out_l)
            right_res.append(out_r)
        return left_res, right_res

def berechne_diff_daten(text1, text2):
    sm = difflib.SequenceMatcher(None, text1, text2)
    opcodes = sm.get_opcodes()
    diff_data = []
    block_id = 0

    for tag, i1, i2, j1, j2 in opcodes:
        block_left = text1[i1:i2]
        block_right = text2[j1:j2]
        ratio = 0
        left_html, right_html = [], []

```

```

        if tag == 'replace':
            str_left = "".join(block_left)
            str_right = "".join(block_right)
            ratio = round(difflib.SequenceMatcher(None, str_left,
str_right).ratio() * 100, 1)
            left_html, right_html = get_inline_diff(block_left,
block_right)
        else:
            left_html = [html.escape(l) for l in block_left]
            right_html = [html.escape(l) for l in block_right]

        diff_data.append({
            'id': block_id,
            'tag': tag,
            'left': left_html,
            'right': right_html,
            'ratio': ratio
        })
        block_id += 1
    return diff_data

class DiffRequestHandler(BaseHTTPRequestHandler):
    def log_message(self, format, *args):
        pass

    def do_GET(self):
        global START_FILE_LEFT, START_FILE_RIGHT
        if self.path == '/':
            self.send_response(200)
            self.send_header('Content-type', 'text/html;
charset=utf-8')
            self.end_headers()

            info_l = get_file_info(START_FILE_LEFT)
            info_r = get_file_info(START_FILE_RIGHT)
            initial_diff = berechne_diff_daten(info_l['content'],
info_r['content'])

            html_content = self.get_html_template().replace(
                "INITIAL_DIFF_DATA", json.dumps(initial_diff)
            ).replace(
                "INITIAL_FILES_LEFT", json.dumps(info_l['siblings'])
            ).replace(
                "INITIAL_FILES_RIGHT", json.dumps(info_r['siblings'])
            ).replace(
                "INITIAL_PATH_LEFT", info_l['path']
            ).replace(
                "INITIAL_PATH_RIGHT", info_r['path']

```

```

        ).replace(
            "APP_VERSION", APP_VERSION
        )
        self.wfile.write(html_content.encode('utf-8'))

elif self.path.startswith('/api/select_file'):
    file_path = waehle_datei_dialog("Wähle eine neue Datei")
    info = get_file_info(file_path) if file_path else {"path":
None}

    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    self.wfile.write(json.dumps(info).encode('utf-8'))

def do_POST(self):
    if self.path == '/api/read_file':
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)
        data = json.loads(post_data.decode('utf-8'))
        info = get_file_info(data['path'])

        self.send_response(200)
        self.send_header('Content-type', 'application/json')
        self.end_headers()
        self.wfile.write(json.dumps(info).encode('utf-8'))

    elif self.path == '/api/diff':
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)
        data = json.loads(post_data.decode('utf-8'))
        diff_data = berechne_diff_daten(data['textLeft'],
data['textRight'])

        self.send_response(200)
        self.send_header('Content-type', 'application/json')
        self.end_headers()
        self.wfile.write(json.dumps(diff_data).encode('utf-8'))

def get_html_template(self):
    return """
<!DOCTYPE html>
<html lang="de">
<head>
    <meta charset="utf-8">
    <title>Advanced Delta Tool</title>
    <style>
        :root {
            --bg-dark: #1e1e1e; --bg-panel: #252526;

```

```

--bg-header: #333333;
    --text-main: #d4d4d4; --accent: #007acc;
    --insert: rgba(40, 167, 69, 0.2); --border-insert:
#28a745;
    --delete: rgba(220, 53, 69, 0.2); --border-delete:
#dc3545;
    --replace: rgba(0, 123, 255, 0.2);
--border-replace: #007bff;
}
body { margin: 0; padding: 0; font-family: 'Segoe UI',
Tahoma, sans-serif; height: 100vh; display: flex; flex-direction:
column; background-color: var(--bg-dark); color: var(--text-main);
overflow: hidden; }

    #header-container { position: sticky; top: 0; z-index:
100; display: flex; background: var(--bg-header); border-bottom: 2px
solid #444; box-shadow: 0 4px 6px rgba(0,0,0,0.3); }
    .pane-header { flex: 1; padding: 10px 15px; display:
flex; flex-direction: column; gap: 8px; min-width: 0; }
    .pane-header.left { border-right: 1px solid #444; }

    .file-controls { display: flex; gap: 10px;
align-items: center; }
    .file-path { font-family: 'Consolas', monospace;
font-size: 13px; color: #9cdcfe; padding: 6px; background: #1e1e1e;
border: 1px dashed #555; border-radius: 4px; cursor: pointer;
white-space: nowrap; overflow: hidden; text-overflow: ellipsis; flex:
1;}
    .file-path:hover { background: #2a2d2e; border-color:
var(--accent); }
    .file-path::before { content: '📄 '; }
    .dir-dropdown { background: #3c3c3c; color: white;
border: 1px solid #555; padding: 5px; border-radius: 3px; max-width:
150px; outline: none; cursor: pointer; }
    .dir-dropdown:focus { border-color: var(--accent); }

    .search-bar { display: flex; gap: 5px; }
    .search-bar input { flex: 1; padding: 4px 8px;
background: #3c3c3c; border: 1px solid #555; color: white;
border-radius: 3px; outline: none; min-width: 50px; }
    .search-bar input:focus { border-color: var(--accent);
}
    .search-bar button { background: #555; border: none;
color: white; padding: 4px 10px; cursor: pointer; border-radius: 3px;
}
    .search-bar button:hover { background: #666; }
    .search-stats { font-size: 12px; align-self: center;
color: #aaa; min-width: 40px; text-align: right;}

```

```

        .delta-nav-panel { width: 120px; flex-shrink: 0;
background: var(--bg-panel); display: flex; flex-direction: column;
justify-content: center; align-items: center; gap: 5px; border-left:
1px solid #444; border-right: 1px solid #444; padding: 5px 0; }
        .delta-nav-title { font-size: 11px; color: #888;
text-transform: uppercase; letter-spacing: 1px; }
        .delta-nav-version { color: var(--accent);
font-weight: bold; }
        .delta-nav-buttons button { background: #555; border:
none; color: white; padding: 4px 12px; cursor: pointer; border-radius:
3px; font-size: 12px; }
        .delta-nav-buttons button:hover { background:
var(--accent); }
        .delta-nav-stats { font-size: 13px; font-weight: bold;
color: #fff; font-family: monospace; margin-top: 2px; }

#workspace { display: flex; flex: 1; overflow: hidden;
position: relative; width: 100%; }

        .editor-container { flex: 1; overflow-y: auto;
overflow-x: auto; font-family: 'Consolas', monospace; font-size: 13px;
line-height: 1.5; padding: 10px 0; scroll-behavior: smooth;
transition: background-color 0.2s; }
        .editor-container::-webkit-scrollbar { width: 12px;
height: 12px; }
        .editor-container::-webkit-scrollbar-thumb {
background: #555; border-radius: 6px; }
        .drag-over { background-color: rgba(0, 122, 204, 0.1)
!important; box-shadow: inset 0 0 10px #007acc; }

#svg-container { width: 120px; flex-shrink: 0;
position: relative; background: var(--bg-panel); border-left: 1px
solid #444; border-right: 1px solid #444; cursor: col-resize; }
        svg { position: absolute; top: 0; left: 0; width:
100%; height: 100%; pointer-events: none; }

#summary-panel { width: 400px; flex-shrink: 0;
background: #1e1e1e; border-left: 2px solid #555; display: flex;
flex-direction: column; transition: width 0.3s; }
        .summary-header { background: #333; padding: 8px;
text-align: center; font-size: 12px; font-weight: bold; border-bottom:
1px solid #444; display: flex; justify-content: space-between;
align-items: center; }
        #summary-content { flex: 1; overflow-y: auto;
overflow-x: hidden; }
        .summary-row { display: flex; border-bottom: 1px solid
#333; cursor: pointer; transition: background-color 0.2s; }

```

```

        .summary-row:hover { background-color: #2a2d2e; }
        .summary-col { flex: 1; overflow: hidden;
text-overflow: ellipsis; white-space: nowrap; padding: 4px;
font-family: 'Consolas', monospace; font-size: 11px; }
        .summary-col.left { border-right: 1px solid #444; }

        .code-block { padding: 2px 10px; white-space: pre;
border-left: 3px solid transparent; border-right: 3px solid
transparent; transition: background-color 0.3s; }
        .tag-equal { color: #d4d4d4; }
        .tag-insert { background-color: var(--insert);
border-right-color: var(--border-insert); }
        .tag-delete { background-color: var(--delete);
border-left-color: var(--border-delete); text-decoration:
line-through; opacity: 0.7; }
        .tag-replace { background-color: var(--replace);
border-left-color: var(--border-replace); border-right-color:
var(--border-replace); }

        .char-diff { font-weight: bold; border-radius: 2px;
padding: 0 1px; }
        .tag-delete .char-diff { background-color: rgba(220,
53, 69, 0.6); color: white; text-decoration: none; }
        .tag-insert .char-diff { background-color: rgba(40,
167, 69, 0.6); color: white; }
        .tag-replace .char-diff { background-color: rgba(0,
123, 255, 0.6); color: white; }

        .active-delta { outline: 2px solid #ff9800;
background-color: rgba(255, 152, 0, 0.2) !important; z-index: 10;
position: relative;}
        .toggle-active { background-color: #dc3545 !important;
border-color: #dc3545 !important; }

        mark.search-hit { background-color: #ffeb3b; color:
black; border-radius: 2px; padding: 0 2px; font-weight: bold;
box-shadow: 0 0 4px #ffeb3b;}
        mark.active-hit { background-color: #ff9800; color:
white; box-shadow: 0 0 6px #ff9800; outline: 1px solid #fff;}

        .percent-label { font-family: sans-serif; font-size:
10px; font-weight: bold; fill: #fff; text-anchor: middle;
dominant-baseline: middle; }
        .label-bg { fill: #007bff; rx: 4; ry: 4; }

        .btn-excel { background-color: #217346; color: white;
border: none; padding: 4px 8px; border-radius: 3px; cursor: pointer;
font-size: 11px; font-weight: bold; display: flex; align-items:

```

```

center; gap: 4px; transition: background-color 0.2s;}
        .btn-excel:hover { background-color: #1e6b3e; }
    </style>
</head>
<body>
    <div id="header-container">
        <div class="pane-header left">
            <div class="file-controls">
                <div class="file-path" id="path-left"
ondblclick="changeFile('left')" title="Doppelklick für lokalen
PC">INITIAL_PATH_LEFT</div>
                <select id="dropdown-left"
class="dir-dropdown" onchange="loadFileFromDropdown('left')"></select>
            </div>
            <div class="search-bar">
                <input type="text" id="search-input-left"
placeholder="Suchen..." onkeyup="if (event.key==='Enter')
executeSearch('left')">
                <button
onclick="executeSearch('left')"><img alt="magnifying glass icon" data-bbox="465 415 480 430"/></button>
                <button onclick="stepSearch('left',
-1)">▲</button>
                <button onclick="stepSearch('left',
1)">▼</button>
                <span class="search-stats" id="stats-left"
style="margin-left:5px">0/0</span>
            </div>
        </div>

        <div class="delta-nav-panel">
            <div class="delta-nav-title">Deltas <span
style="color:var(--accent)">APP_VERSION</span></div>
            <div class="delta-nav-buttons">
                <button onclick="stepDelta(-1)">▲</button>
                <button onclick="stepDelta(1)">▼</button>
            </div>
            <button id="toggle-deltas-btn"
onclick="toggleDeltas()" style="width: 90%; background:#007acc;
color:white; border:none; padding:3px; font-size:10px;
border-radius:3px; cursor:pointer;">Nur Deltas</button>
            <div class="delta-nav-stats"
id="delta-stats-display">0/0</div>
        </div>

        <div class="pane-header right">
            <div class="file-controls">
                <div class="file-path" id="path-right"
ondblclick="changeFile('right')" title="Doppelklick für lokalen

```



```

PC">INITIAL_PATH_RIGHT</div>
        <select id="dropdown-right"
class="dir-dropdown"
onchange="loadFileFromDropdown('right') "></select>
        </div>
        <div class="search-bar">
            <input type="text" id="search-input-right"
placeholder="Suchen..." onkeyup="if(event.key==='Enter')
executeSearch('right') ">
            <button
onclick="executeSearch('right') ">🔍</button>
            <button onclick="stepSearch('right',
-1) ">▲</button>
            <button onclick="stepSearch('right',
1) ">▼</button>
            <span class="search-stats" id="stats-right"
style="margin-left:5px">0/0</span>
        </div>
    </div>
    <div id="workspace">
        <div id="left-editor" class="editor-container"></div>
        <div id="svg-container"><svg
id="lines-svg"></svg></div>
        <div id="right-editor" class="editor-container"></div>

        <div id="summary-panel">
            <div class="summary-header">
                <span style="flex: 1; text-align: left;
margin-top: 3px;">ZUSAMMENFASSUNG</span>
                <div style="display: flex; gap: 10px;">
                    <button class="btn-excel"
onclick="exportExcel()" title="Als .xls für Microsoft Excel
exportieren">📄 EXCEL</button>
                    <button onclick="toggleSummary()"
style="background:none; border:none; color:#aaa; cursor:pointer;
font-size: 14px;">✖</button>
                </div>
            </div>
            <div id="summary-content"></div>
        </div>
    </div>

    <script>
        // --- State ---
        let diffData = INITIAL_DIFF_DATA;
        let filesLeft = INITIAL_FILES_LEFT;

```

```

    let filesRight = INITIAL_FILES_RIGHT;

    let deltaBlocks = [];
    let currentDeltaIndex = -1;
    let showOnlyDeltas = false;
    let currentZoom = 13;
    let isSummaryVisible = true;

    const leftEditor =
document.getElementById('left-editor');
    const rightEditor =
document.getElementById('right-editor');
    const svgContainer =
document.getElementById('svg-container');
    const svg = document.getElementById('lines-svg');

    function updateDropdown(side, siblings, currentPath) {
        const select =
document.getElementById(`dropdown-${side}`);
        select.innerHTML = '<option value="">-- Im Ordner
--</option>';
        siblings.forEach(f => {
            const opt = document.createElement('option');
            opt.value = f.path; opt.textContent = f.name;
            if (f.path === currentPath) opt.selected =
true;
            select.appendChild(opt);
        });
    }

    function initDeltas() {
        deltaBlocks = diffData.filter(b => b.tag !==
'equal');
        currentDeltaIndex = -1;

document.getElementById('delta-stats-display').textContent =
deltaBlocks.length > 0 ? `0/${deltaBlocks.length}` : "0/0";
    }

    function stepDeltaToId(targetId) {
        const idx = deltaBlocks.findIndex(b => b.id ===
targetId);
        if (idx !== -1) {
            currentDeltaIndex = idx - 1;
            stepDelta(1);
        }
    }

```

```

        function stepDelta(direction) {
            if (deltaBlocks.length === 0) return;

            if (currentDeltaIndex >= 0 && currentDeltaIndex <
deltaBlocks.length) {
                const oldId =
deltaBlocks[currentDeltaIndex].id;

document.getElementById(`left-block-${oldId}`)?.classList.remove('acti
ve-delta');

document.getElementById(`right-block-${oldId}`)?.classList.remove('act
ive-delta');
            }

            currentDeltaIndex += direction;
            if (currentDeltaIndex >= deltaBlocks.length)
currentDeltaIndex = 0;
            if (currentDeltaIndex < 0) currentDeltaIndex =
deltaBlocks.length - 1;

            const targetId =
deltaBlocks[currentDeltaIndex].id;
            const leftEl =
document.getElementById(`left-block-${targetId}`);
            const rightEl =
document.getElementById(`right-block-${targetId}`);

            if (leftEl) leftEl.classList.add('active-delta');
            if (rightEl)
rightEl.classList.add('active-delta');

            if (leftEl) leftEl.scrollIntoView({ behavior:
'smooth', block: 'center' });
            else if (rightEl) rightEl.scrollIntoView({
behavior: 'smooth', block: 'center' });

document.getElementById('delta-stats-display').textContent =
`${currentDeltaIndex + 1}/${deltaBlocks.length}`;
        }

        function toggleDeltas() {
            showOnlyDeltas = !showOnlyDeltas;
            const btn =
document.getElementById('toggle-deltas-btn');

            if (showOnlyDeltas) {

```

```

        btn.classList.add('toggle-active');

document.querySelectorAll('.tag-equal').forEach(el => el.style.display
= 'none');
        } else {
            btn.classList.remove('toggle-active');

document.querySelectorAll('.tag-equal').forEach(el => el.style.display
= 'block');
        }

        if
(document.getElementById('search-input-left').value)
executeSearch('left');
        if
(document.getElementById('search-input-right').value)
executeSearch('right');

        setTimeout(drawLines, 50);
    }

    function toggleSummary() {
        isSummaryVisible = !isSummaryVisible;
        const panel =
document.getElementById('summary-panel');
        panel.style.width = isSummaryVisible ? '400px' :
'0px';
        panel.style.borderLeft = isSummaryVisible ? '2px
solid #555' : 'none';
        setTimeout(drawLines, 300);
    }

    // --- Excel Export Fix ---
    function exportExcel() {
        const pathLeft =
document.getElementById('path-left').textContent;
        const pathRight =
document.getElementById('path-right').textContent;

        let htmlTable = `<html
xmlns:o="urn:schemas-microsoft-com:office:office"
xmlns:x="urn:schemas-microsoft-com:office:excel"
xmlns="http://www.w3.org/TR/REC-html40">
        <head>
            <meta charset="utf-8">
            <style>
                table { border-collapse: collapse;
font-family: 'Consolas', 'Courier New', monospace; font-size: 12px; }

```

```

        th { background-color: #333333; color:
#ffffff; padding: 6px; text-align: left; border: 1px solid #777777;
font-weight: bold;}
        td { border: 1px solid #cccccc; padding:
4px 6px; vertical-align: top; mso-number-format: "\\@"; }
    </style>
</head>
<body>
    <table>
        <tr>
            <th>Original (Links):<br><span
style="font-weight: normal; font-size: 10px;">${pathLeft}</span></th>
            <th>Geändert (Rechts):<br><span
style="font-weight: normal; font-size: 10px;">${pathRight}</span></th>
        </tr>`;

```

```

diffData.forEach(block => {
    if (block.tag === 'equal') return;

    let bgLeft = 'transparent';
    let bgRight = 'transparent';
    let charStyleLeft = '';
    let charStyleRight = '';

    if (block.tag === 'delete') {
        bgLeft = '#f8d7da';
        charStyleLeft = 'background-color:
#dc3545; color: white; font-weight: bold; text-decoration:
line-through;';
    } else if (block.tag === 'insert') {
        bgRight = '#d4edda';
        charStyleRight = 'background-color:
#28a745; color: white; font-weight: bold;';
    } else if (block.tag === 'replace') {
        bgLeft = '#cce5ff';
        bgRight = '#cce5ff';
        charStyleLeft = 'background-color:
#007bff; color: white; font-weight: bold;';
        charStyleRight = 'background-color:
#007bff; color: white; font-weight: bold;';
    }
}

```

```

    let leftText =
block.left.join('').replace(/\n/g, '<br>');
    let rightText =
block.right.join('').replace(/\n/g, '<br>');

```

```

    leftText = leftText.replace(</span

```

```

class='char-diff'/>/g, `

```

```

    }

    // --- Rendering ---
    function renderEditors() {
        leftEditor.innerHTML = ''; rightEditor.innerHTML =
        '';

        const summaryContent =
        document.getElementById('summary-content');
        summaryContent.innerHTML = '';

        diffData.forEach(block => {
            const leftDiv = document.createElement('div');
            leftDiv.id = `left-block-${block.id}`;
            leftDiv.className = `code-block tag-${block.tag}`;
            leftDiv.innerHTML = block.left.length > 0 ?
            block.left.join('') : '\\n'.repeat(block.right.length);
            if(block.tag === 'insert')
            leftDiv.style.opacity = '0';
            if(showOnlyDeltas && block.tag === 'equal')
            leftDiv.style.display = 'none';
            leftEditor.appendChild(leftDiv);

            const rightDiv =
            document.createElement('div');
            rightDiv.id = `right-block-${block.id}`;
            rightDiv.className = `code-block tag-${block.tag}`;
            rightDiv.innerHTML = block.right.length > 0 ?
            block.right.join('') : '\\n'.repeat(block.left.length);
            if(block.tag === 'delete')
            rightDiv.style.opacity = '0';
            if(showOnlyDeltas && block.tag === 'equal')
            rightDiv.style.display = 'none';
            rightEditor.appendChild(rightDiv);

            if (block.tag !== 'equal') {
                const row = document.createElement('div');
                row.className = 'summary-row';
                row.onclick = () =>
                stepDeltaToId(block.id);

                const sl = document.createElement('div');
                sl.className = `summary-col left
                tag-${block.tag}`;
                sl.innerHTML = block.left.length > 0 ?
                block.left.join('') : ' ';

                const sr = document.createElement('div');
                sr.className = `summary-col

```

```
tag-${block.tag}`;
                                sr.innerHTML = block.right.length > 0 ?
block.right.join('') : ' ';

                                row.appendChild(sl); row.appendChild(sr);
                                summaryContent.appendChild(row);
                            }
                        });

                        initDeltas();
                        clearSearch('left'); clearSearch('right');
                        setTimeout(drawLines, 50);
                    }

function drawLines() {
    svg.innerHTML = '';
    const svgWidth = svgContainer.clientWidth;
    const leftScroll = leftEditor.scrollTop;
    const rightScroll = rightEditor.scrollTop;

    diffData.forEach(block => {
        if (block.tag === 'equal') return;
        const leftEl =
document.getElementById(`left-block-${block.id}`);
        const rightEl =
document.getElementById(`right-block-${block.id}`);
        if (!leftEl || !rightEl) return;

        const leftY = leftEl.offsetTop +
(leftEl.offsetHeight / 2) - leftScroll;
        const rightY = rightEl.offsetTop +
(rightEl.offsetHeight / 2) - rightScroll;

        let color = '#007bff';
        if (block.tag === 'insert') color = '#28a745';
        if (block.tag === 'delete') color = '#dc3545';

        const path =
document.createElementNS('http://www.w3.org/2000/svg', 'path');
        const curve = `M 0 ${leftY} C ${svgWidth/2}
${leftY}, ${svgWidth/2} ${rightY}, ${svgWidth} ${rightY}`;

        path.setAttribute('d', curve);
        path.setAttribute('fill', 'none');
path.setAttribute('stroke', color);
        path.setAttribute('stroke-width', '2');
path.setAttribute('opacity', '0.6');
        svg.appendChild(path);
```



```

        if (block.tag === 'replace') {
            const midY = (leftY + rightY) / 2; const
midX = svgWidth / 2;

            const rect =
document.createElementNS('http://www.w3.org/2000/svg', 'rect');
            rect.setAttribute('x', midX - 20);
rect.setAttribute('y', midY - 10);
            rect.setAttribute('width', '40');
rect.setAttribute('height', '20');
            rect.setAttribute('class', 'label-bg');
svg.appendChild(rect);

            const text =
document.createElementNS('http://www.w3.org/2000/svg', 'text');
            text.setAttribute('x', midX);
text.setAttribute('y', midY + 1);
            text.setAttribute('class',
'percent-label');

            text.textContent = `${block.ratio}%`;
svg.appendChild(text);
        }
    });
}

    async function loadFileFromDropdown(side) {
        const select =
document.getElementById(`dropdown-${side}`);
        const path = select.value;
        if(!path) return;

        const response = await fetch('/api/read_file', {
method: 'POST', body: JSON.stringify({path: path}) });
        const data = await response.json();

document.getElementById(`path-${side}`).textContent = data.path;
        let tL = (side === 'left') ? data.content :
extractLeftText();
        let tR = (side === 'right') ? data.content :
extractRightText();

        const diffRes = await fetch('/api/diff', { method:
'POST', body: JSON.stringify({textLeft: tL, textRight: tR}) });
        diffData = await diffRes.json();
        renderEditors();
    }
}

```

```

        async function changeFile(side) {

document.getElementById(`path-${side}`).textContent = "Lade
Dateiauswahl...";

        const response = await fetch('/api/select_file');
        const data = await response.json();

        if (data.path) {

document.getElementById(`path-${side}`).textContent = data.path;
        updateDropdown(side, data.siblings,
data.path);

        let tL = (side === 'left') ? data.content :
extractLeftText();
        let tR = (side === 'right') ? data.content :
extractRightText();

        const diffRes = await fetch('/api/diff', {
method: 'POST', body: JSON.stringify({textLeft: tL, textRight: tR})
});

        diffData = await diffRes.json();
        renderEditors();

        }
    }

    function extractLeftText() { let r=[];
diffData.forEach(b => { r.push...(b.raw_left || b.left)); }); return
r; }

    function extractRightText() { let r=[];
diffData.forEach(b => { r.push...(b.raw_right || b.right)); });
return r; }

    // --- Drag & Drop ---
    function preventDefaults(e) { e.preventDefault();
e.stopPropagation(); }
    ['dragenter', 'dragover', 'dragleave',
'drop'].forEach(evt => {
        leftEditor.addEventListener(evt, preventDefaults,
false); rightEditor.addEventListener(evt, preventDefaults, false);
    });
    ['dragenter', 'dragover'].forEach(evt => {
        leftEditor.addEventListener(evt, () =>
leftEditor.classList.add('drag-over'), false);
        rightEditor.addEventListener(evt, () =>
rightEditor.classList.add('drag-over'), false);
    });
    ['dragleave', 'drop'].forEach(evt => {

```

```

        leftEditor.addEventListener(evt, () =>
leftEditor.classList.remove('drag-over'), false);
        rightEditor.addEventListener(evt, () =>
rightEditor.classList.remove('drag-over'), false);
    });
    leftEditor.addEventListener('drop', (e) =>
handleDrop(e, 'left'), false);
    rightEditor.addEventListener('drop', (e) =>
handleDrop(e, 'right'), false);

    function handleDrop(e, side) {
        const file = e.dataTransfer.files[0];
        if (!file) return;
        const reader = new FileReader();
        reader.onload = async function(event) {
            let content =
event.target.result.match(/.*(?:\\r\\n|\\r|\\n)|.+$/g) || [];

document.getElementById(`path-${side}`).textContent = `[Drag & Drop]
${file.name}`;

document.getElementById(`dropdown-${side}`).innerHTML = '<option>--
Datei gezogen --</option>';

            let tL = (side === 'left') ? content :
extractLeftText();
            let tR = (side === 'right') ? content :
extractRightText();

            const diffRes = await fetch('/api/diff', {
method: 'POST', body: JSON.stringify({textLeft: tL, textRight: tR})
});
            diffData = await diffRes.json();
            renderEditors();
        };
        reader.readAsText(file);
    }

    // --- View-Based Search Logic ---
    let searchState = { left: { hits: [], currentIndex: -1
}, right: { hits: [], currentIndex: -1 } };
    function escapeRegExp(string) { return
string.replace(/[\.*+?^${}()|\\[\]\\\]/g, '\\\\$&'); }

    function clearSearch(side) {
        const editor =
document.getElementById(`${side}-editor`);
        const marks =

```

```

Array.from(editor.querySelectorAll('mark.search-hit'));
        marks.forEach(mark => {
            const parent = mark.parentNode;

parent.replaceChild(document.createTextNode(mark.textContent), mark);
            parent.normalize();
        });
        searchState[side] = { hits: [], currentIndex: -1
    };

document.getElementById(`stats-${side}`).textContent = "0/0";
    }

    function highlightNode(node, regex, hits) {
        if (node.nodeType === 3) {
            let match = node.nodeValue.match(regex);
            if (match) {
                const mark =
document.createElement('mark');
                mark.className = 'search-hit';

                const splitNode =
node.splitText(match.index);
                const remainderNode =
splitNode.splitText(match[0].length);

mark.appendChild(splitNode.cloneNode(true));
                splitNode.parentNode.replaceChild(mark,
splitNode);

                hits.push(mark);

                highlightNode(remainderNode, regex, hits);
            }
        } else if (node.nodeType === 1 && node.nodeName
!== 'MARK') {
            const children = Array.from(node.childNodes);
            children.forEach(child => highlightNode(child,
regex, hits));
        }
    }

    function executeSearch(side) {
        clearSearch(side);
        const query =
document.getElementById(`search-input-${side}`).value;
        if (!query) return;

```

```

        const editor =
document.getElementById(`${side}-editor`);
        const regex = new RegExp(escapeRegExp(query),
'i');

editor.querySelectorAll('.code-block').forEach(block => {
    if (block.style.opacity === '0' ||
block.style.display === 'none') return;
    highlightNode(block, regex,
searchState[side].hits);
});

    if (searchState[side].hits.length > 0) {
        stepSearch(side, 1);
    } else {

document.getElementById(`stats-${side}`).textContent = "0/0";
    }
}

function stepSearch(side, direction) {
    const state = searchState[side];
    if (state.hits.length === 0) return;

    if (state.currentIndex >= 0 && state.currentIndex
< state.hits.length) {

state.hits[state.currentIndex].classList.remove('active-hit');
    }

    state.currentIndex += direction;
    if (state.currentIndex >= state.hits.length)
state.currentIndex = 0;
    if (state.currentIndex < 0) state.currentIndex =
state.hits.length - 1;

    const targetMark = state.hits[state.currentIndex];
    targetMark.classList.add('active-hit');
    targetMark.scrollIntoView({ behavior: 'smooth',
block: 'center' });

document.getElementById(`stats-${side}`).textContent =
`${state.currentIndex + 1}/${state.hits.length}`;
    }

    // --- Synchronisation & Events ---

```

```

        let isSyncingLeft = false; let isSyncingRight = false;
        leftEditor.addEventListener('scroll', function() {
            if (!isSyncingLeft) {
                isSyncingRight = true;
                const percentage = this.scrollTop /
(this.scrollHeight - this.clientHeight);
                rightEditor.scrollTop = percentage *
(rightEditor.scrollHeight - rightEditor.clientHeight);
                drawLines();
            }
            isSyncingLeft = false;
        });
        rightEditor.addEventListener('scroll', function() {
            if (!isSyncingRight) {
                isSyncingLeft = true;
                const percentage = this.scrollTop /
(this.scrollHeight - this.clientHeight);
                leftEditor.scrollTop = percentage *
(leftEditor.scrollHeight - leftEditor.clientHeight);
                drawLines();
            }
            isSyncingRight = false;
        });

        let isDragging = false;
        svgContainer.addEventListener('mousedown', () => {
isDragging = true; document.body.style.cursor = 'col-resize';
document.body.style.userSelect = 'none'; });
        document.addEventListener('mousemove', (e) => {
            if (!isDragging) return;
            const containerRect =
document.getElementById('workspace').getBoundingClientRect();
            const mouseX = e.clientX - containerRect.left;
            const totalWidth = containerRect.width;
            const leftFlex = (mouseX - 60) / totalWidth; const
rightFlex = 1 - leftFlex - (120 / totalWidth);
            if (leftFlex > 0.1 && rightFlex > 0.1) {
leftEditor.style.flex = leftFlex; rightEditor.style.flex = rightFlex;
drawLines(); }
        });
        document.addEventListener('mouseup', () => { if
(isDragging) { isDragging = false; document.body.style.cursor =
'default'; document.body.style.userSelect = 'auto'; } });

        window.onload = () => {
            updateDropdown('left', filesLeft,
document.getElementById('path-left').textContent);
            updateDropdown('right', filesRight,

```

```

document.getElementById('path-right').textContent);
        renderEditors();
    };
    window.addEventListener('resize', drawLines);
</script>
</body>
</html>
"""

def start_app():
    global START_FILE_LEFT, START_FILE_RIGHT
    print(f"Starte Advanced Delta Tool {APP_VERSION}...")

    START_FILE_LEFT = waehle_datei_dialog("Wähle die LINKE Datei")
    if not START_FILE_LEFT: return
    START_FILE_RIGHT = waehle_datei_dialog("Wähle die RECHTE Datei")
    if not START_FILE_RIGHT: return

    port = 8080
    server = HTTPServer(('localhost', port), DiffRequestHandler)
    print(f"\nServer läuft auf http://localhost:{port}")
    webbrowser.open(f'http://localhost:{port}')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nServer beendet.")
        server.server_close()

if __name__ == "__main__":
    start_app()

```